

LOG430 - Étape 1

Nom: Ludovic Marcoux Hoyos
Groupe: 02
Session: Été 2025

Description

Ce projet est une application gérant le stock, les commandes, les transferts, les clients et les ventes d'une compagnie.

L'architecture

Architecture du système

Le système est conçu selon une architecture à base de microservices conteneurisés via Docker Compose, favorisant la modularité, l'évolutivité et la scalabilité horizontale.

1. Couche `shared_data`

Chaque microservice gère sa propre propriété d'accès aux données via SQLAlchemy, connecté à une base PostgreSQL commune.

- **Modèles :**
 - `Produit`, `Vente`, `Client`, `Panier`, `ProduitPanier`, etc.
 - Organisés dans chaque microservice (`stock_service`, `customer_service`, etc.)
- **DAO :**
 - Chaque entité dispose d'un DAO (ex: `ProduitDAO`) avec des méthodes comme `get_by_id`, `rechercher`, `update`, etc.

2. Services

Cette couche encapsule la logique métier spécifique à chaque microservice.

- `StockService`
- `RestockService`
- `ReportingService`
- `CustomerService`
- `CartService`

Chaque service fait appel à ses DAO pour appliquer la logique d'affaires.

3. Couche `API` (FastAPI)

Chaque microservice expose une API REST documentée via Swagger (OpenAPI 3.0), et utilise :

- Authentification par `X-API-Key`
- Logging structuré
- Instrumentation Prometheus via `prometheus_fastapi_instrumentator`
- Middleware CORS

4. API Gateway (KrakenD)

- Centralise toutes les routes REST via `krakend.json`
- Transmet les en-têtes (`X-API-Key`) aux microservices backend
- Compatible avec Swagger UI pour une interface unifiée

5. Observabilité

- **Prometheus** : collecte les métriques depuis chaque microservice
- **Grafana** : visualisation des dashboards (latence, disponibilité, instance)
- **Requêtes Prometheus** :
 - `histogram_quantile(0.95, rate(http_request_duration_seconds_bucket[1m]))`
 - `rate(http_requests_total{status!~"4..|5.."}[1m])`
 - `sum by (instance) (rate(http_requests_total[1m]))`

Besoins fonctionnels et non-fonctionnels

☒ Besoins fonctionnels

Le système permet la gestion de stock, la vente, le panier client et le reporting via une interface REST.

1. Gestion du stock

- `GET /stock/` : liste des produits
- `POST /stock/vente/annulation`
- `PUT /stock/update/:produit_id/magasin/:magasin_id`

2. Réapprovisionnement

- `POST /restock/transférer`
- `GET /restock/stock_par_magasin`

3. Tableau de bord / Reporting

- `GET /reporting/ventes, /ruptures, /dashboard`

4. Gestion client

- `POST /clients/, GET /clients/{client_id}`

5. Panier

- `POST /panier/, /panier/{id}/produit, /panier/{id}/checkout`
 - `GET /panier/panier/{id}`
-

☑ Besoins non-fonctionnels

🔗 Architecture

- Microservices FastAPI
- Base de données PostgreSQL commune
- Communication via Docker internal network

🔍 Testabilité

- Tests unitaires des DAO et services
- Tests d'intégration des routes FastAPI

📦 Observabilité

- Prometheus scrappe `/metrics` de chaque service
- Dashboards Grafana pour :
 - Latence 95e percentile
 - Taux de réussite des requêtes
 - Répartition de charge

🚀 Performance

- Load testing avec `k6`
- API Gateway avec KrakenD (future option : Kong)
- Scalabilité horizontale testée avec `cart_service_1` et `cart_service_2`

📅 Déploiement

- Docker Compose (multi-service)
- Script d'initialisation de la base (`init_db.py`, `seed_db.py`)

📖 Swagger unifié

- Swagger UI connecté à KrakenD via `swagger-config.yaml`

Justification des décisions d'architecture (ADR)

1. Choix de la base de données – ADR

Statut : Accepté Date : 2025-07-16

Décision : Utilisation de PostgreSQL comme base de données relationnelle commune aux microservices.

Contexte : Le système devait évoluer vers une architecture distribuée avec plusieurs services (stock, panier, clients, reporting...). Une base centralisée et robuste devenait nécessaire pour permettre la cohérence des

données.

Choix possibles :

SQLite : trop limité pour une architecture distribuée (verrouillages, accès concurrents).

MongoDB : NoSQL mal adapté à la structure relationnelle (produits, clients, ventes).

PostgreSQL : puissant, open-source, supporte transactions, relations, JSON.

Conséquences :

Chaque microservice se connecte à une base PostgreSQL partagée.

Permet l'intégrité référentielle (clé étrangère client_id, produit_id, etc.).

Simplifie les tests avec des conteneurs postgres isolés.

2. Séparation des responsabilités – ADR

Statut : Accepté Date : 2025-07-16

Décision : Adoption d'une architecture microservices avec séparation par domaine fonctionnel.

Contexte : Les cas d'utilisation critiques UC1–UC7 nécessitaient des évolutions indépendantes (ajout du panier, création de clients, reporting, réapprovisionnement...). L'architecture en couches n'était plus suffisante seule.

Choix possibles :

Application monolithique : difficilement maintenable à long terme.

Modularité par classes/modules : limite les déploiements indépendants.

Architecture microservices + API Gateway : flexible, déployable, extensible.

Conséquences :

Un service = un domaine métier (stock, panier, reporting, etc.).

Facilité de scaling horizontal.

Possibilité de résilience par service.

Choix technologiques



Pourquoi ? Framework Python moderne, asynchrone, basé sur OpenAPI.

Avantage : Génère automatiquement la documentation Swagger, compatible avec Pydantic et les outils modernes de monitoring.

PostgreSQL

Pourquoi ? SGBD robuste, mature, SQL, avec bonne scalabilité verticale.

Avantage : Relations fortes entre entités, transactions ACID, index efficaces.

Redis

Pourquoi ? Caching rapide pour améliorer les temps de réponse des endpoints critiques (/dashboard, /stock/create...).

Avantage : Stockage clé/valeur en RAM, TTL, utilisé avec aioredis.

Architecture microservices + API Gateway

Pourquoi ? Favorise l'indépendance des déploiements et la résilience.

Avantage : Chaque service peut être développé, testé et déployé indépendamment.

KrakenD (ou Kong) comme API Gateway

Pourquoi ? Centralise les appels, permet de filtrer, authentifier et agréger les requêtes.

Avantage : Supporte les en-têtes (ex: X-API-Key), la validation, CORS, Swagger unifié.

Docker + Docker Compose

Pourquoi ? Isolation, portabilité, simplicité de déploiement.

Avantage : Une commande (make up) démarre l'ensemble de l'environnement avec tous les services et observabilité.

Prometheus + Grafana

Pourquoi ? Suivre la latence, l'erreur et le débit de chaque service.

Avantage : Visualisation fine de l'utilisation, alerting possible, intégration avec prometheus_fastapi_instrumentator.

pytest

Pourquoi ? Plus flexible que unittest, riche en plugins.

Avantage : Permet des tests unitaires et d'intégration efficaces, facilement intégrable dans un pipeline CI/CD.

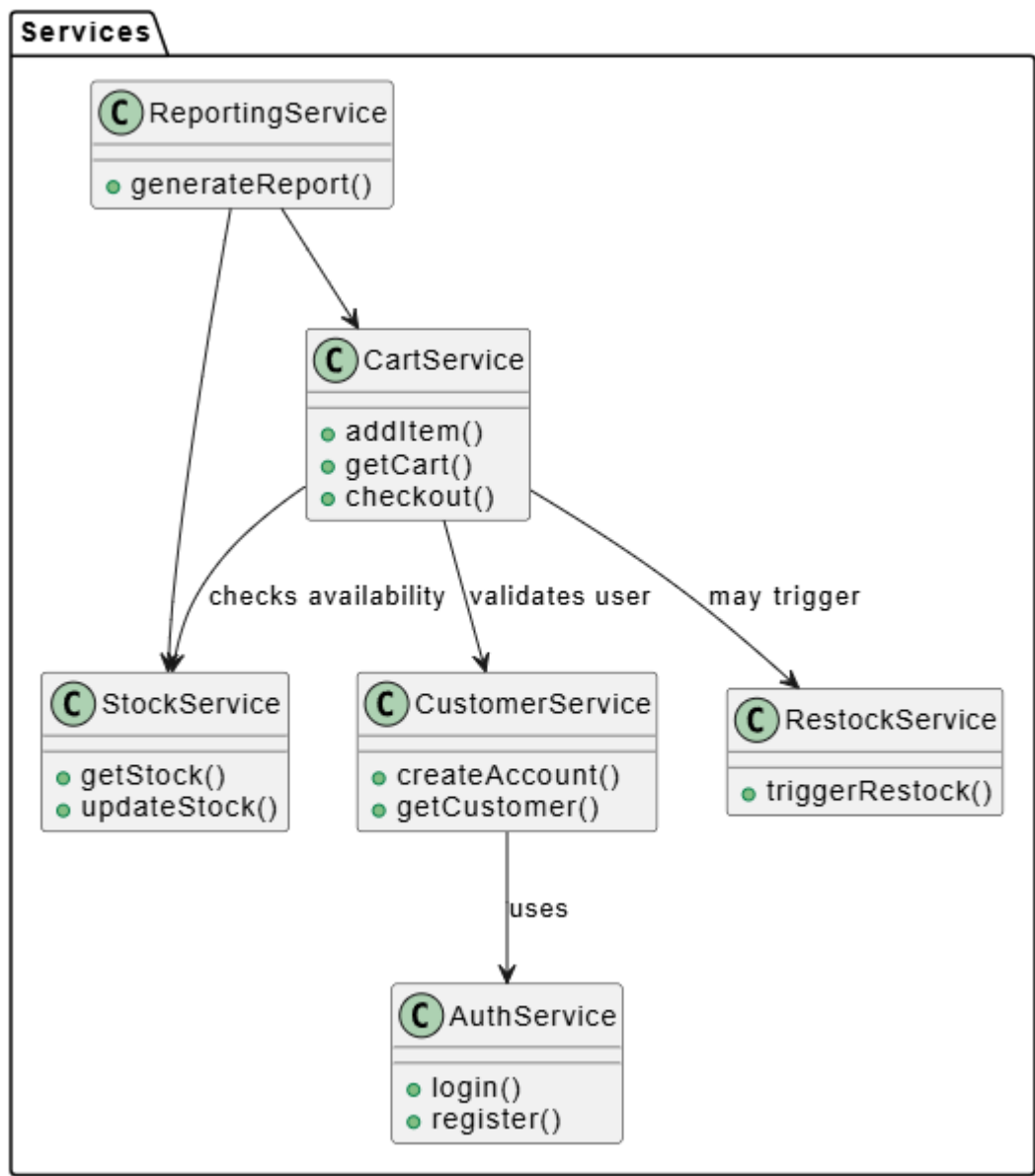
GitHub Actions ou GitLab CI

Pourquoi ? Intégration continue, déploiement automatique possible.

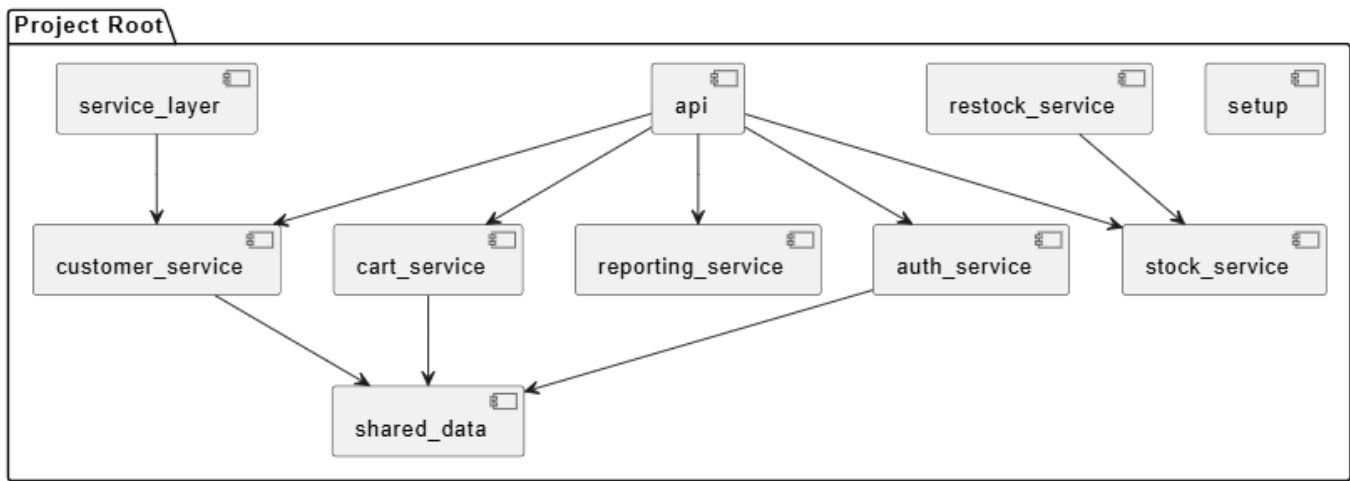
Avantage : Qualité assurée avant merge, pipeline reproductible et traçable.

Diagrammes

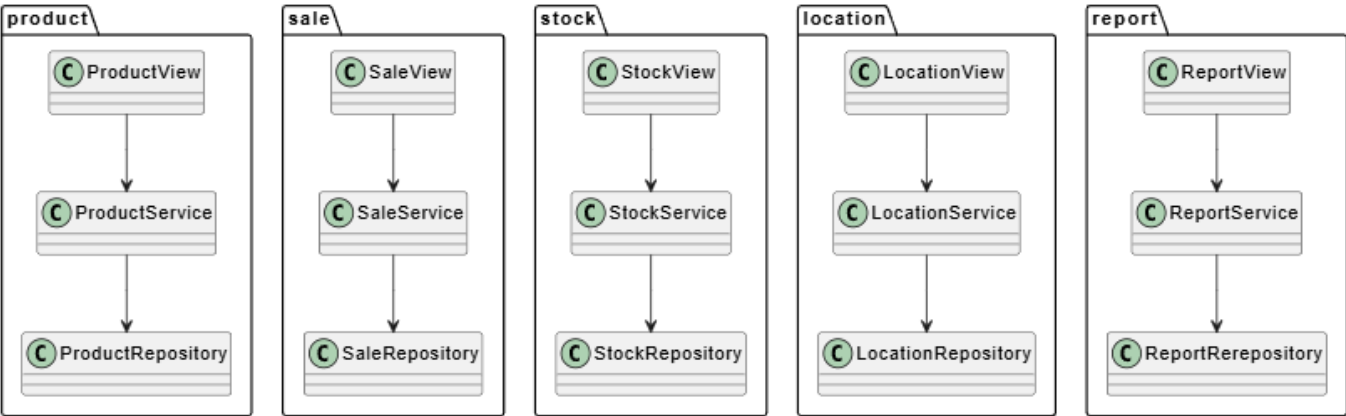
Vue Logique



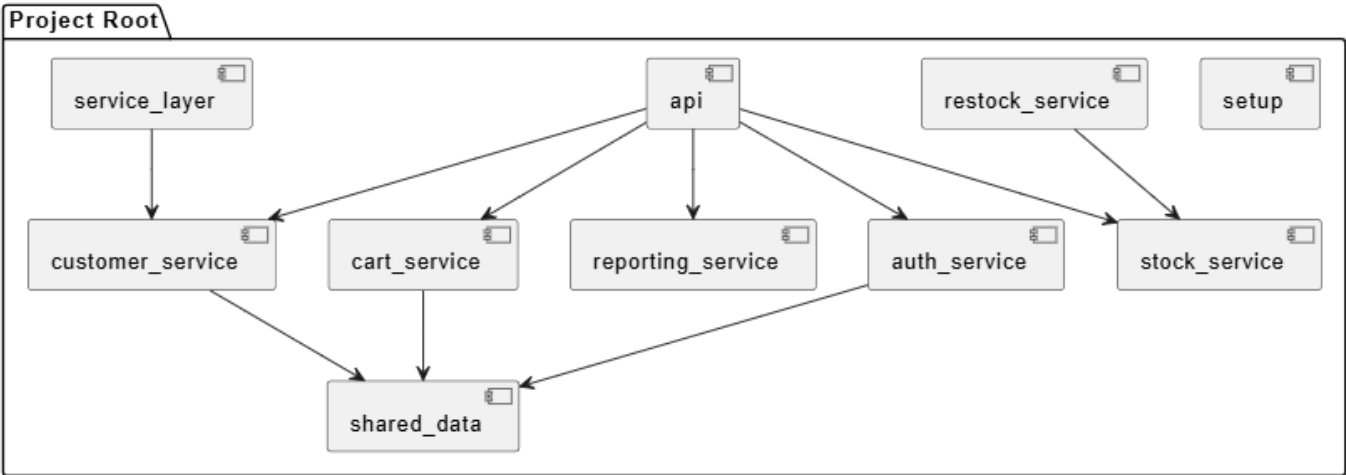
Vue d'implémentation



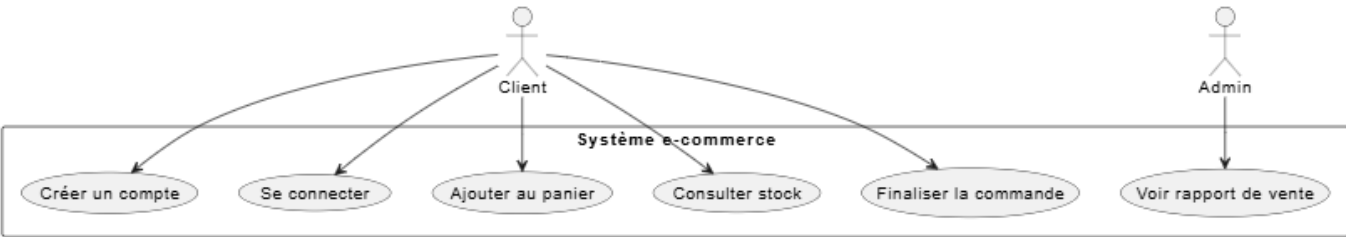
Vue de déploiement



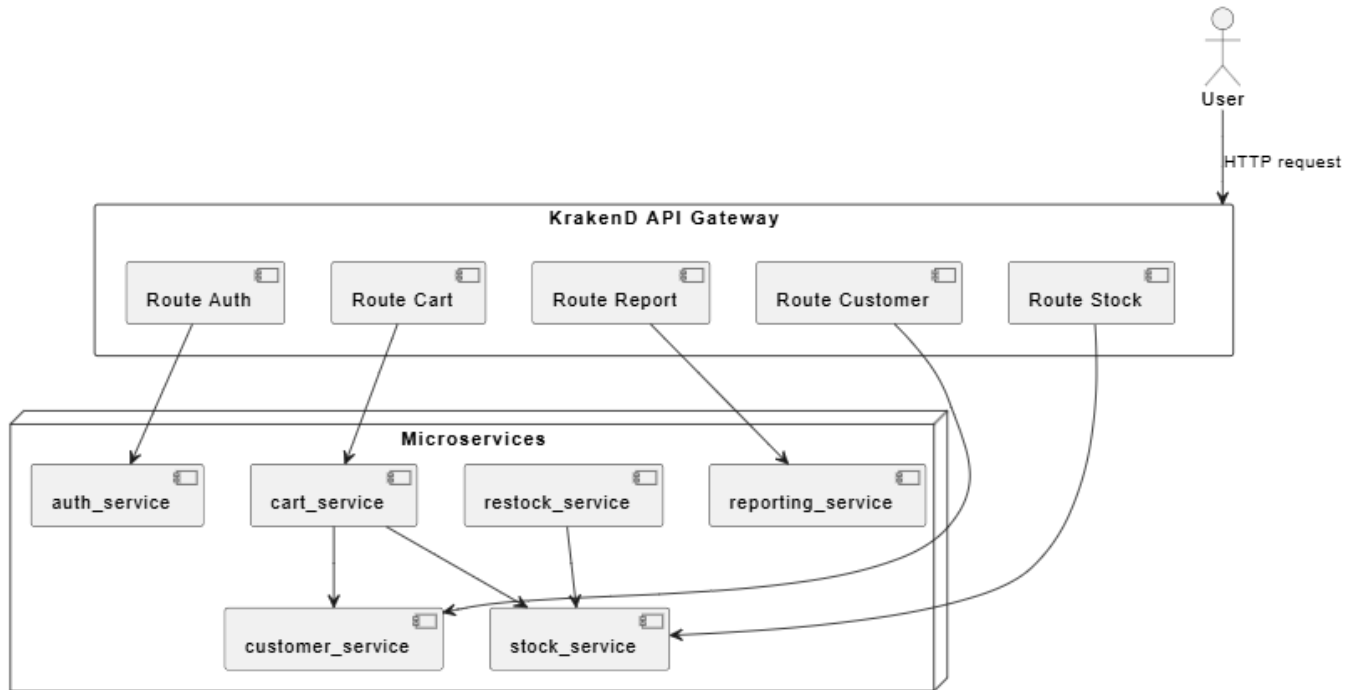
Vue d'implémentation



Vue de cas d'utilisation



Diagrammes de séquence (processus)



Instruction d'installation et d'execution

Cloner

Git bash: `git clone https://github.com/Ludo67/LOG430-Lab0.git`

Installer .venv.

Installer un .venv. voir (<https://packaging.python.org/en/latest/guides/installing-using-pip-and-virtual-environments/>)

Installer dépendances

`pip install -r requirements.txt`

setup projet(terminal)

`make build make init-db make seed-db make up`

Executer les tests unitaires

Terminal: `make test- + nom du service`

Instruction pour l'environnement de production

Dans la machine virtuelle, voici des commandes à utiliser.

Télécharger la plus nouvelle version sur docker hub

`docker pull ludo678/my-app:latest`



CI/CD Pipeline – Description des étapes

Ce pipeline GitHub Actions automatise les étapes de **linting**, **tests**, **build Docker**, et **publication** sur Docker Hub lors d'un **push** ou **pull_request** sur n'importe quelle branche.

Déclencheurs

```
on:
  push:
    branches: [ "*" ]
  pull_request:
    branches: [ "*" ]
```